



Module Code & Module Title

CS4051NI\CC4059NI Fundamental of Computing

70% Individual Coursework

Submission: Milestone 1

Academic Semester: Spring Semester 2026

Credit: 15 credit semester long module

Student Name: Arjan Upadhayaya Timalisina

London Met ID: 25029344

College ID: NP01NT4A250057

Google Link:

https://drive.google.com/drive/folders/1Zrdrxa-iAOY4CfA9_r-_3KaUd0bx9pC8?usp=sharing

Assignment Due Date: Friday, April 24, 2026

Assignment Submission Date: Friday, April 24, 2026

I confirm that I understand my coursework needs to be submitted online via MST Classroom under the relevant module page before the deadline for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

Table of Contents

Introduction.....	2
Overview.....	2
Aims and Objectives.....	2
Technologies Used.....	3
Algorithm.....	5
Flow Chart.....	6
Data Structure.....	7
1. Primitive Data Type.....	7
a. Strings.....	7
b. Integers.....	7
c. Floats.....	7
2. Collection Data Type.....	8
a. Lists.....	8
b. Dictionaries.....	8
Program Outline.....	9
1. Program Structure.....	9
2. Program Modules.....	9
a. Reading Function (read_inventory).....	9
b. Display Function (display_inventory).....	9
c. Main Section.....	9
3. Program Output.....	10
Conclusion.....	11
References.....	12

Introduction

Overview

MedStore Pvt. Ltd. is a wholesale medicine distributor that manages bulk stock across a range of medicines from various brands. To streamline its day to day operations, this project presents a console based Medicine Management System built in Python. The application automates two core business processes: recording sales to customers and restocking inventory from suppliers. It reads medicine data from a structured text file, presents the available stock to the administrator, processes transactions interactively, generates uniquely named invoice and note files for each transaction, and immediately updates the inventory file to reflect real-time stock levels.

The system applies business rules such as a 5% strip discount when a customer purchases two or more strips of the same medicine, and supports multi-item transactions so that a single invoice can cover purchases of multiple medicine types in one session.

Aims and Objectives

The primary aim of this project is to develop a functional, modular, and reliable medicine wholesale management application that replaces manual record-keeping with an automated, file-driven system.

The specific objectives are:

1. To read and parse medicine inventory data from a plain text file and display it in a readable, formatted table.
2. To implement a sale transaction module that allows the administrator to sell medicines per tablet or per strip, applies the 5% bulk discount where applicable, supports multi-item carts, and generates a VAT/sale invoice as a `.txt` file.

3. To implement a restock transaction module that allows existing medicines to be replenished or new medicines to be added to inventory, and generates a corresponding restock note.
 4. To update the inventory file immediately after every transaction so that stock levels are always current and accurate.
 5. To validate all user inputs and display meaningful error messages when invalid data is entered, ensuring the program does not crash or produce incorrect results.
 6. To ensure each generated invoice or note carries a unique timestamped filename to prevent overwriting and allow easy record retrieval.
 7. To keep the program running in a continuous loop until the administrator explicitly chooses to exit.
-

Technologies Used

The following technologies and tools were used in the development of this project:

1. Python 3 — The primary programming language used to build the entire application, chosen for its readability, rich standard library, and strong support for file I/O and data structures.
2. Python Standard Library (`os`, `datetime`) — Used for file existence checks, reading and writing inventory and invoice files, and generating timestamped filenames.
3. Plain Text Files (`.txt`) — Used both as the persistent inventory database (`medicines.txt`) and as the output format for all generated invoices and restock notes.
4. Python Data Structures — Dictionaries were used to represent individual medicine records, and lists were used to hold the full inventory and transaction

carts, enabling efficient lookup, iteration, and in-place updates.

5. Modular Programming — The application is structured into clearly separated functions for file I/O, display, input validation, sale handling, restock handling, and invoice generation, following good software design principles.

Algorithm

Step 1. START

Step 2. Identify the medicine inventory file (e.g., 'medicines.txt').

Step 3. Check if 'medicines.txt' exists.

Step 3.1: If file does not exist, display error message and STOP.

Step 4. Open 'medicines.txt' for reading.

Step 5. Read file line by line:

Step 5.1: Strip leading/trailing spaces from each line.

Step 5.2: If line is empty, skip to next line.

Step 5.3: Split line by comma into pieces.

Step 5.4: If number of pieces is not as expected, display warning and skip.

Step 5.5: Parse pieces as:

- name (string)
- brand (string)
- stock (integer)
- price per tablet (float)
- price per strip (float)
- tablets per strip (integer)

Step 5.6: If parsing fails, display warning and skip.

Step 5.7: If parsing succeeds, store medicine info in a dictionary and add to medicines list.

Step 6. After reading all lines, check if medicines list is empty.

Step 6.1: If empty, display message and STOP.

Step 7. Display inventory heading.

Step 8. Print column headers.

Step 9. For each medicine in medicines list:

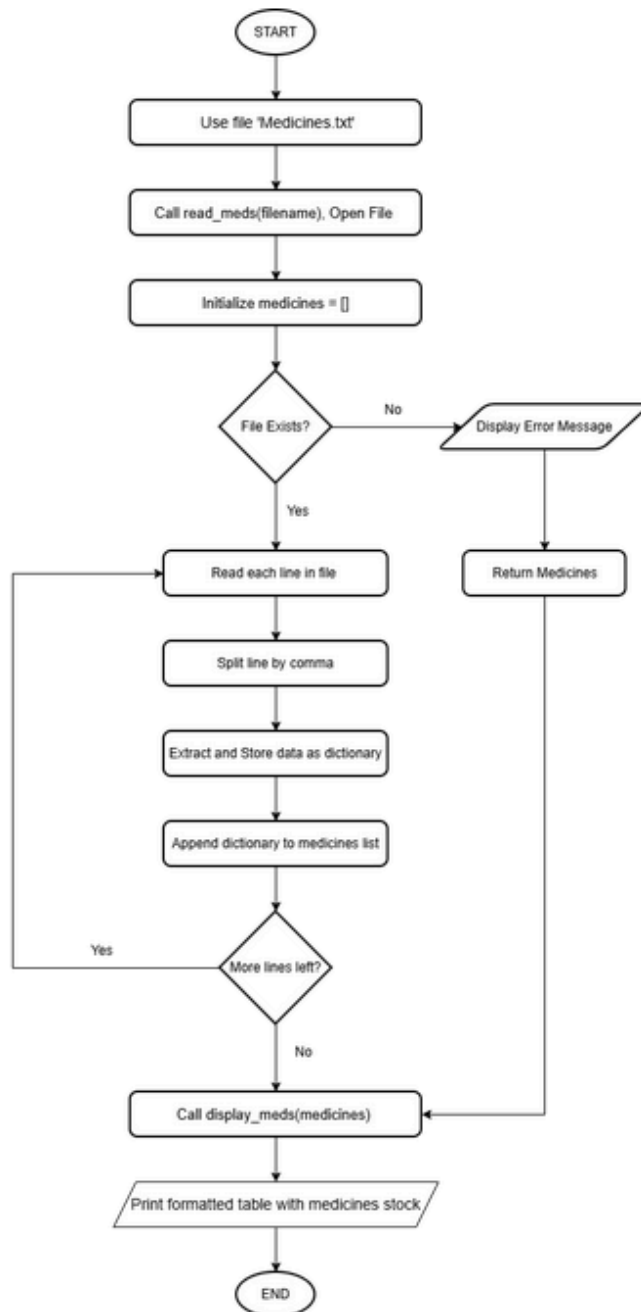
Step 9.1: Calculate number of strips = stock // tablets per strip.

Step 9.2: Print medicine details in a formatted row, including number of strips.

Step 10. Print a line to mark end of inventory display.

Step 11. END

Flow Chart



Data Structure

1. Primitive Data Type

a. Strings

In the provided code, strings are used extensively to handle textual information. They represent file paths such as "medicines.txt", medicine names, brands, and are also used when reading and processing each line from the inventory file. Strings are essential for both storing and displaying textual data throughout the program.

Example: "Paracetamol", "Cipla", "medicines.txt"

b. Integers

Integers are employed to represent whole-number values. These include the number of tablets in stock ('stock_tablets'), the number of tablets per strip ('tablets_strip'), as well as counters like the line number when reading the file and the index when displaying the inventory. Integers ensure that quantities and counts are handled accurately.

Example: '100', '10', '1'

c. Floats

Floats are used for numeric values that may contain decimals, specifically for the rates of medicines per tablet ('rate_tablet') and per strip ('rate_strip'). This allows the program to handle prices and rates with the necessary precision for financial calculations.

Example: '1.5', '15.0'

2. Collection Data Type

a. Lists

Lists serve as ordered, mutable collections in the code. The 'medicines' list accumulates each medicine's data as a dictionary, representing the entire inventory. Additionally, the 'parts' list temporarily holds the split components of each line from the file, making it easier to parse and validate the data before storing it.

Example: ["Paracetamol", "Cipla", "100", "1.5", "15.0", "10"],
[medicine1_dict, medicine2_dict]

b. Dictionaries

Dictionaries are used to store structured information about each medicine. Each dictionary contains key-value pairs for attributes such as the medicine's name, brand, stock count, rates, and packaging details. This structure allows for organized storage and easy access to each medicine's data. All such dictionaries are collected in the 'medicines' list for further processing and display.

Example: {"name": "Paracetamol", "brand": "Cipla", "stock_tablets": 100, "rate_tablet": 1.5, "rate_strip": 15.0, "tablets_strip": 10}

Program Outline

1. Program Structure

This program is developed in Python using a modular approach, organizing functionality into separate functions for clarity and maintainability. All logic is contained in a single file (medshow.py), but the code is structured into distinct functions for reading and displaying the medicine inventory. The program uses both primitive data types (strings, integers, floats) and collection types (lists, dictionaries). The inventory data is stored externally in a text file named 'medicines.txt'.

2. Program Modules

a. Reading Function (read_inventory)

The read_inventory function is responsible for handling the 'medicines.txt' file. It checks for the file's existence, reads each line, strips whitespace, splits the line by commas, and converts the values into appropriate data types. If a line is malformed or contains invalid data, a warning is displayed and the line is skipped. Valid medicine records are stored as dictionaries in a list.

b. Display Function (display_inventory)

The display_inventory function presents the medicine stock in a formatted table. It prints a heading, column headers, and then iterates through the list of medicines, displaying each medicine's details in a neat row, including a calculated value for the number of strips available. If no medicines are found, it displays an appropriate message.

c. Main Section

The main section of the program calls the reading and displaying functions in sequence. It first reads the inventory from the file, then displays the inventory if any medicines are found. This section ensures the program executes the correct workflow when run as a script.

3. Program Output

When run, the program displays the medicine inventory in a formatted table. It shows a heading with the store name, column headers, and a row for each medicine with details like name, brand, stock, prices, tablets per strip, and calculated strips available. If the inventory file is missing or empty, an error or "no medicines found" message is shown. The output ends with a line marking the end of the display.

Conclusion

In conclusion, this Python program is a simple way to keep track of medicines in a store. It reads information about each medicine from a text file, checks if the data is correct, and then shows everything in a clear table. If there are any problems, like missing files or wrong data, the program gives helpful error messages so the user knows what went wrong.

The code uses lists and dictionaries to organize the medicine data, which makes it easy to add new features later, such as searching for a medicine or updating the stock. The program also separates the part that reads the data from the part that displays it, making the code easier to understand and change.

The way the table is printed makes it easy for anyone to see important details, like how many tablets are in stock and how many strips can be made. This is useful for people who need to quickly check the inventory.

Overall, this project is a good example for students learning Python. It shows how to read and write files, handle errors, and display information in a neat way. By starting with this code, students can practice their skills and try adding more features to make the program even better.

References

1. [Python Official Documentation](#)
2. [Reading and Writing Files — Python Docs](#)
3. [String Methods — Python Docs](#)
4. [Errors and Exceptions — Python Docs](#)
5. [Formatted String Literals \(f-strings\) — Python Docs](#)
6. [Data Structures — Python Docs](#)
7. [os.path.exists — Python Docs](#)